Code Inspection Report Response

IWAC Conference Application for Heather Falconer

By: PenUltimate

Brett Palmer, Monica Agneta, George Pitt, Rebecca Sonnemann, Ben Caras

23 March 2026

Table of Contents

1 Introduction……..……...…….…2

1.1 Purpose……..…….…….….2

1.2 References…….……...…….…….…..2

2 CIR Response…..…….…….…….…..3

2.1 Coding & Commenting…….…….…..…3

2.2 Defects……...…….…….…….…...3

2.2.1 Navigation and Application Layout Module...……....3

2.2.2 Authentication Module……..……..……4

2.2.3 Panel Module……..……..……...4

2.2.4 Messaging Module……..……....5

2.2.5 Profile Module…….……..……..……6

2.2.6 Notifications Module……..……..……...6

2.2.7 Miscellaneous…….……..……...6

3 Future Actions and Takeaways……7

Appendix A – Team and Client Review Sign off……....……8

Appendix B – Contributions……...…….9

1. Introduction

This capstone project is being completed in partial fulfillment of the requirements for the B.S. in Computer Science degree for the University of Maine. The client for this project is part of the Association for Writing Across the Curriculum (AWAC), which organizes the International Writing Across the Curriculum Conference (IWAC), a multi-day event that hosts educators and researchers. In 2027, UMaine is hosting this conference. The client has requested the development of a mobile application that provides a way for in-person and remote attendees to develop a personal agenda of sessions, access presentation materials, and receive conference updates. Furthermore, the application should enhance the conference experience by allowing users to create profiles, message one another, and comment on sessions. Overall, the project aims to provide an intuitive platform that enhances the conference experience for all IWAC 2027 attendees.

1.1 Purpose of the Document

The purpose of this Code Inspection Report (CIR) Response document is to formally address the feedback and critiques identified during the code inspection conducted by Thread Zeppelin for the IWAC Conference Application. This document provides a structured response to each identified issue, either by refuting the critique with supporting evidence or by acknowledging its validity and outlining a plan for remediation. The CIR Response creates transparency between the PenUltimate development team and the client Dr. Falconer about how code quality concerns will be addressed by the project's close in May. It also establishes accountability within the PenUltimate development team by assigning each developer to corrective actions and deadlines, and reinforcing coding standards and project requirements.

1.2 References

Thread Zeppelin. (2026). *Code inspection report*.

PenUltimate (2025). *IWAC Conference App*. GitHub. https://github.com/umaine-capstone-iwac/IWAC-Conference-App

2. CIR Response

The input from Thread Zeppelin's Code Inspection Report has been analyzed to ensure the highest code quality and functionality. The development team has taken the feedback regarding each of our modules seriously, and wrote this document to outline the intended modifications meant to improve readability, functionality, and security. This process has effectively communicated our resolution of the comments provided in the Code Inspection Report.

2.1 Coding & Commenting Conventions

As a team we chose automated formatting tools such as Prettier and ESLint to enforce consistent coding conventions across all files. These tools ensure that formatting rules are applied automatically, reducing the need for manual checks of different contributors. During early stages of development, we divided coding work by features of the application so that each member could work on separate screens simultaneously. Since our team is small and frequently met, we communicated directly rather than creating formal pull requests and detailed comments. Later on as the application progressed, we made sure that each function included at least one comment. Regarding comment placement, we recognize that comments can be found both above and inline depending on the function. Moving forward we will make the preferred commenting convention to place descriptive comments above the function to improve consistency across the files.

2.2 Defects

This section quotes each of the defects listed in section 4 of the Code Inspection Report written by Thread Zeppelin, with each defect grouped into different sections based on which module it corresponds to. Any defects that need to be fixed will be acknowledged along with a resolution on how they will be fixed, or otherwise refuted in the case that it is an intentional feature, rather than an overlooked error.

2.2.1 Navigation and Application Layout Module

- "The repository still contains the Expo scaffold reset script. The file itself states that it can be safely deleted after setup. Leaving it in the production repository is not recommended."
    - This critique is valid, and the file reset-project.js has been accordingly deleted.

2.2.2 Authentication Module

- "Check for branching or validation logic that allows invalid states and/or fails valid ones."
    - As of right now both the login and create account screens do not have the functionality from attempting multiple sign in attempts simultaneously. This will be fixed by including the use of the setIsLoading useState.
- "Check for user inputs that are not being checked for empty values, invalid formats, excessive length, or unsafe content."
    - The user create account doesn't check all user input fields. This will be fixed by including user input error handling for all user input fields.
- "Check for fields required by the specification but not enforced in the UI or database workflow."
    - The user create account and login requires that all user input fields are not empty. During the Code Inspection Review certain input fields didn't require you to fill them in. These errors have since been fixed.
- "The forgot-password flow is only partially implemented. In the code, pressing Forgot Password sends a reset email using a deep-link redirect (iwacapp://resetpassword), but during inspection it was reported that this feature did not fully work in practice and was still in progress. The implementation is also platform dependent."
    - The forgot password functionality has since been completed and works on both iOS and Android. The "platform dependent" miscommunication may have come from the fact that it must be tested in a development build instead of Expo Go due to the reset password deep linking.

2.2.3 Panel Module

- "This shared component appears to contain direct Supabase/database calls. That creates tighter coupling between UI rendering and data access which makes the component harder to test in isolation"
    - The shared component for panel details directly makes its own database calls as it represents a full feature of the panel expansion that a user is navigated to when they click on a conference panel. Since the data is tightly coupled to the panel being displayed, it makes sense to keep the Supabase queries within the component so that all logic is together. It was decided to design the panelDetials.tsx file as a feature-level shared component to remove duplicated logic in the index.tsx file and sessions.tsx files. Consolidating both the UI and associated data fetching ensures that panel detail updates only need to be made in one file.

- “Terminology appears inconsistent across the app and notes, with"event ","session " and "panel " sometimes used interchangeably. This is a minor flaw and should probably be discussed with their client.”
    - The terminology in the application reflects distinctions made by the client for the structure of the conference. The term “session” refers to a specific time block in the conference schedule. For example, Session A represents the 9-10:30am time period, while a “panel” refers to an individual presentation within that time block. The term “event” appeared earlier in development as another way to discuss the panels. During later revisions, files were updated so that the code consistently uses only “session” and “panel”.
- “In sessions.tsx, events are grouped with const key = r.session; and the grouped map is keyed only by the session string. If session labels can repeat across days, this could incorrectly combine events from different dates into one group.”
    - The code pulls the sessions from the database and adds the sessions to a bucket if they are the same. Sessions labels would not repeat across days.

2.2.4 Messaging Module

- “Any user can message any other user.”
    - Our requirement was for users to message other conference attendees. Since this application is for a paid professional conference, our client was not concerned with users abusing the messaging feature. She confirmed that any issues could be reported directly to her outside of the application.
- “The preview list logic in messages.tsx involves multiple dependent queries, fetching messages, deriving unique partner IDs, then fetching user and profile data for each partner, creating several points where data could be missing, mis-ordered, or stale.”
    - The data within the messages table is created on the sending of a message, with fields such as user IDs, content, and timestamp being automatically generated. These fields are non-nullable in our database, so if they were missing for any reason, the insert would fail. Therefore, we do not have to worry about missing data on queries for the “messages” table.
- “There did not appear to be any real-time update mechanism, meaning a user would not see new messages without manually refreshing.”
    - The Conversation screen does contain a functional real-time update mechanism. The function responsible for this has the header “REALTIME SUBSCRIPTION” with descriptive comments throughout such as “Subscribe to new messages in this conversation.”
- “Our inspection noted concerns about sorting by timestamp and unread message handling, both of which are easy to get subtly wrong.”
    - While the message sorting has currently been tested as working, we acknowledge that this feature has many edge cases. We have attempted to mitigate such issues

by using the “timestamptz” data type to ensure consistency across timezones, and having is_read default to False and be set to True once a conversation is opened by the recipient. To ensure the feature works as expected, Brett will add a filterMessages unit test to our DevOps by 3/25.

2.2.5 Profile Module

- “No protections were observed against inappropriate biography text or problematic uploaded profile images. Again, this is technically okay with their client, but it is a content-safety concern.”
    - While it can potentially be a content-safety concern, it was again confirmed that our client does not find this addition necessary, as all attendees will almost certainly be professionals within this field who have registered for this event. Because of this, it is in everyone’s best interest to be professional with what details they put in their profile.

- “The profile settings page allows editing first name, last name, profession, about me, interests, and my sessions, then writes them straight to Supabase. There is no visible validation for blank values, excessive length, inappropriate text, or file-type/size constraints on the avatar beyond image-picker selecting images.”
    - There is no requirement for users to actually provide their interests or personal information on their profile, so it shouldn’t be an issue if they leave it blank, which will simply show a box with the corresponding title (such as “about me”), and no text under it. Conversely, if they wish to write a lot, there’s no reason to not allow them. When it comes to the avatar, image-picker ensures that they upload an image, and beyond that, image size should not be an issue.

2.2.6 Notifications Module

- “The notification settings checkboxes appear to change only local UI state and do not persist settings or affect actual notification behavior. It was also reported that they cannot be tested on the web, which is a minor flaw right now.”
    - Rather than having a page for the notification settings, the “manage notifications” button simply takes you to your phone’s native settings page for our app. Because of this, the “notification settings” screen will not be utilized in the actual app.

2.2.7 Miscellaneous

- “Some unused starter assets such as default images (like React logo) appear to remain in the project. Deleting them would make the repository cleaner.”
    - This critique is valid, and all files and images from the initial React Native boilerplate application have been removed.

3. Future Actions and Takeaways

We appreciate the feedback provided in the CIR written by the Thread Zeppelin capstone team. This document helped us draft a clear plan to remove critical defects in our app by the project's completion in May. On March 11th PenUltimate met to review all of the defects, and assigned a responsible team member and deadline to those that were valid. Developers will complete all necessary changes and fixes the week of March 23rd, then PenUltimate will reconvene on March 29th to perform a code review for validation. Furthermore, on March 24th PenUltimate will be meeting with our client Dr. Falconer to clarify any potential defects related to the system requirements. We will schedule an acceptance with her after March 29th to validate all features that were altered due to the CIR. Finally, team members will also be responsible for writing unit tests for their corresponding code sections by the end of March. The CIR was particularly useful in highlighting which test cases are critical to address in this process.

Appendix A – Team and Client Review Sign-off

All team members and the client have signed to acknowledge they have reviewed this document and agreed on both its content and format. If the team members or the client have any comments, they may state them below.

Rebecca Sonnemann

Signature: Rebecca Sonnemann
Rebecca Sonnemann (Mar 23, 2026 19:22:51 EDT) Date: 3/23/2026

Comments: _____

Brett Palmer

Signature: Brett Palmer
Brett Palmer (Mar 23, 2026 19:19:47 EDT) Date: 3/23/2026

Comments: _____

Monica Agneta

Signature: Monica Agneta
Monica Agneta (Mar 23, 2026 19:24:11 EDT) Date: 3/23/2026

Comments: _____

Ben Caras

Signature: Benjamin Caras
Benjamin Caras (Mar 23, 2026 19:22:04 EDT) Date: 3/23/2026

Comments: _____

George Pitt

Signature: George Pitt
George Pitt (Mar 23, 2026 19:21:23 EDT) Date: 3/23/2026

Comments: _____

Heather Falconer

Signature: _____ Date: 3/23/2026

Comments: _____

Appendix B – Document Contributions

Listed below are the contributions from each team member, and which sections of the document they worked on.

Brett

Brett wrote the introduction, section 1.1, and section 2.2.1, 2.2.4, and 2.2.7.

Monica

Monica wrote section 2.1 and did ⅔ of section 2.2.3.

George

George wrote the introductions for sections 2 and 2.2, as well as sections 2.2.5, and 2.2.6.

Rebecca

Rebecca formatted the document, created the table of contents, and did ⅓ of section 2.2.3.

Ben

Ben wrote section 2.2.2 and section 3.